Dossier d'Architecture Technique & Spécifications Système

Plateforme Web Officielle & Système d'Information — Division Génie Infotronique (GIT)

---

1. Stack Technologique Globale

Le socle technique est conçu pour offrir une haute performance, une maintenance simplifiée et une intégration fluide avec l'infrastructure réseau locale de la division.

Composant	Technologie retenue	Description & Usage
Framework Full-Stack	Next.js (App Router)	React framework avec Server Actions et API Routes unifiées
Langage	TypeScript	Typage statique strict sur l'ensemble du projet (front & back)
Styling & UI	Tailwind CSS v4	Framework CSS utility-first couplé à Lucide Icons & Framer Motion
ORM & Base de données	Prisma ORM	Couche d'accès aux données type-safe avec MySQL / PostgreSQL
Authentification & RBAC	NextAuth.js / Auth.js	Gestion des sessions JWT & rôles (`MEMBER`, `TREASURER`, `CENSOR`, `LAB_MANAGER`, `ADMIN`)
Scanner QR Code	`@zxing/library`	Module d'acquisition et lecture QR Code via la WebCam / Caméra mobile

Composant	Technologie retenue	Description & Usage
Intégration Réseau	Node.js Client RouterOS	Client API REST / Socket pour le contrôle d'accès Mikrotik & UniFi

---

2. Modèle de Données & Schéma Prisma (`schema.prisma`)

Le schéma ci-dessous définit les entités principales, la gestion des accès, la comptabilité interne, les sanctions disciplinaires ainsi que la traçabilité des événements.datasource db {

provider = "mysql"

url = env("DATABASE_URL")

}

generator client {

provider = "prisma-client-js"

}

enum Role {

MEMBER

TREASURER

CENSOR

LAB_MANAGER

ADMIN

}

enum PenaltyStatus {

PENDING

```
  PAID
  CANCELLED
}
enum EventStatus {
  UPCOMING
  ONGOING
  COMPLETED
}
enum CourseStatus {
  PLANNED
  IN_PROGRESS
  COMPLETED
}
model User {
  id            String         @id @default(uuid())
  email         String         @unique
  passwordHash  String
  fullName      String
  matricule     String?        @unique
  phone         String?
  role          Role           @default(MEMBER)
```

```
  qrCodeToken    String          @unique @default(uuid())
  macAddress     String?         @unique
  isWifiAllowed Boolean          @default(false)
  isLabAllowed  Boolean          @default(false)
  createdAt      DateTime        @default(now())
  updatedAt      DateTime        @updatedAt
  contributions ProjectContributor[]
  cotisations    Cotisation[]
  penalties      Penalty[]       @relation("MemberPenalties")
  issuedPenalties Penalty[]     @relation("IssuedByCensor")
  certifications Certification[]
  registrations EventRegistration[]
}

model Project {
  id           String              @id @default(uuid())
  title        String
  slug         String              @unique
  description  String              @db.Text
  githubUrl    String?
  demoUrl      String?
  coverImage   String?
```

```
  createdAt    DateTime            @default(now())
  updatedAt    DateTime            @updatedAt
  contributors ProjectContributor[]
}

model ProjectContributor {
  id           String   @id @default(uuid())
  projectId    String
  userId       String
  roleInProject String?
  project      Project  @relation(fields: [projectId], references: [id], onDelete: Cascade)
  user         User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([projectId, userId])
}

model Cotisation {
  id       String   @id @default(uuid())
  userId   String
  amount   Float
  month    Int
  year     Int
  paidAt   DateTime @default(now())
  scannedBy String
```

```
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)

}

model Penalty {

  id          String        @id @default(uuid())

  userId      String

  censorId    String

  reason      String

  amount      Float

  status      PenaltyStatus @default(PENDING)

  createdAt   DateTime      @default(now())

  paidAt      DateTime?

  user        User          @relation("MemberPenalties", fields: [userId], references: [id], onDelete:
Cascade)

  censor      User          @relation("IssuedByCensor", fields: [censorId], references: [id])

}

model Event {

  id          String           @id @default(uuid())

  title       String

  description String           @db.Text

  startDate   DateTime

  endDate     DateTime?
```

```
  location    String?
  status      EventStatus      @default(UPCOMING)
  coverImage  String?
  gallery     GalleryItem[]
  registrations EventRegistration[]
}

model EventRegistration {
  id        String   @id @default(uuid())
  eventId   String
  userId    String
  createdAt DateTime @default(now())
  event     Event    @relation(fields: [eventId], references: [id], onDelete: Cascade)
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([eventId, userId])
}

model GalleryItem {
  id        String   @id @default(uuid())
  eventId   String
  url       String
  type      String   // IMAGE ou VIDEO
  caption   String?
```

```
  event     Event    @relation(fields: [eventId], references: [id], onDelete: Cascade)
}

model Course {
  id          String       @id @default(uuid())
  title       String
  description String       @db.Text
  status      CourseStatus @default(PLANNED)
  startDate   DateTime?
  certifications Certification[]
}

model Certification {
  id        String   @id @default(uuid())
  courseId  String
  userId    String
  issuedAt  DateTime @default(now())
  course    Course   @relation(fields: [courseId], references: [id], onDelete: Cascade)
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([courseId, userId])
}
```

---

3. Architecture des Routes API

Les points de terminaison REST et Server Actions sont segmentés par domaine fonctionnel avec vérification systématique des rôles (RBAC).

Authentification & Profil

- `POST /api/auth/login` : Authentification de l'utilisateur et génération du jeton JWT.
- `GET /api/me` : Récupération du profil courant, du jeton QR Code dynamique et des adresses MAC enregistrées.

Administration & Opérations Scanner

- `POST /api/treasury/cotisation` : Enregistrement d'une cotisation mensuelle via scan QR (Paramètres : `userId`, `month`, `year`, `amount`).
- `POST /api/discipline/penalty` : Application d'une pénalité disciplinaire par le Censeur via scan QR (Paramètres : `userId`, `reason`, `amount`).
- `POST /api/treasury/pay-penalty` : Validation du règlement d'une pénalité (Paramètre : `penaltyId`).
- `POST /api/lab/check-access` : Contrôle d'accès physique au laboratoire et validation du droit Wi-Fi (Paramètre : `userId`).
- `POST /api/network/whitelist` : Synchronisation du statut réseau d'un membre vers le contrôleur d'accès.

---

4. Algorithme de Synchronisation Wi-Fi (Mikrotik API Hook)

Le contrôle d'accès au réseau local du laboratoire repose sur l'état financier et disciplinaire de l'utilisateur.

Étape 1 : Évaluation des conditions d'éligibilité

Un utilisateur est considéré comme éligible au réseau Wi-Fi si et seulement si l'ensemble des règles suivantes est vérifié :

- **Cotisation à jour** : `Cotisation.month == CurrentMonth` && `Cotisation.year == CurrentYear`
- **Discipline conforme** : `COUNT(Penalties WHERE status == PENDING) == 0`
- **Matériel enregistré** : `User.macAddress != null`

Étape 2 : Exécution du Hook de synchronisation

+------------------+ Vérification +--------------------+

| Scan / Trigger | ----------------⟶ | Base de Données |



1. **Cas d'éligibilité validée** :
    - Envoi d'une requête REST/API vers le routeur Mikrotik : `/ip/hotspot/ip-binding/add` ou `/ip/dhcp-server/lease/make-static`.
    - L'adresse MAC reçue est passée en état `bypassed` ou enregistrée avec un bail DHCP statique.
    - Mise à jour de la base de données : `User.isWifiAllowed = true`.
2. **Cas d'inéligibilité ou révocation** :
    - Suppression de la règle ou passage en mode `blocked` sur le routeur Mikrotik via l'identifiant MAC.
    - Mise à jour de la base de données : `User.isWifiAllowed = false`.